PROG6112 (Programming 1B) Master Syllabus & Exam Study Guide

SECTION 1: CORE CONCEPTS & CHEAT SHEET

1. The Scanner Buffer Trap

When reading numbers followed by `String` input:

Java

```
int age = scanner.nextInt();
scanner.nextLine(); // CONSUME THE NEWLINE BUFFER!
String name = scanner.nextLine();
```

2. String Parsing (Crucial for GUIs & File I/O)

Java

```
// Convert String to Number
int age = Integer.parseInt("25");
double price = Double.parseDouble("49.99");

// Convert Number to String
String str = String.valueOf(price);
```

3. Key String Methods

Java

```
str.equals("SUV");                // Exact match (NEVER
use == for Strings!)
str.equalsIgnoreCase("suv");      // Case-insensitive
match
str.trim();                        // Removes leading/
trailing spaces
str.substring(0, 3);               // Extracts characters
from index 0 up to 3
str.toUpperCase();                 // Converts to
uppercase
```

4. Integer Division Bug (Average Calculations)

Java

```
int total = 2200;
```

```
int count = 6;

// WRONG: Truncates decimal values (gives 366.0 instead
of 366.67)
double avg = total / count;

// CORRECT: Cast one operand to double
double avg = (double) total / count;
```

5. Formatting Output (`printf` / `String.format`)

- `%-15s` : Left-aligned string (15 width)
- `%8d` : Right-aligned integer (8 width)
- `%.2f` : Floating-point number formatted to 2 decimal places

SECTION 2: FULL WORKED EXAM QUESTION (QUESTION 2 PAPER TEMPLATE)

Below is a complete implementation based on the standard exam format for **Inheritance, Abstract Classes, Interfaces, and Driver Classes**.

PDF

Question Requirements

1	1	Interface `iTickets` with method `print_tickets()`.
2	2	Abstract Class `Tickets` storing fields, constructor, and getters.
3	3	Subclass `TicketSales` implementing discount logic (10% discount if age ≥65).
4	4	Driver class `Movie_Tickets` collecting user input via `Scanner`.

Code Implementation

1. `iTickets.java`

Java

```
public interface iTickets {
    public void print_tickets();
}
```

2. `Tickets.java`

Java

```
public abstract class Tickets implements iTickets {
    private String customerName;
    private String movieTitle;
    private int customerAge;
    private double moviePrice;

    // Constructor
    public Tickets(String customerName, String
movieTitle, int customerAge, double moviePrice) {
        this.customerName = customerName;
        this.movieTitle = movieTitle;
        this.customerAge = customerAge;
        this.moviePrice = moviePrice;
    }

    // Getters (Used by Subclass due to private access)
    public String getCustomerName() {
        return customerName;
    }

    public String getMovieTitle() {
        return movieTitle;
    }

    public int getCustomerAge() {
        return customerAge;
    }

    public double getMoviePrice() {
        return moviePrice;
    }
}
```

3. TicketSales.java

Java

```
public class TicketSales extends Tickets {

    public TicketSales(String customerName, String
movieTitle, int customerAge, double moviePrice) {
        super(customerName, movieTitle, customerAge,
moviePrice);
```

```
    }

    @Override
    public void print_tickets() {
        double discount = 0.0;

        // Apply 10% discount if customer age is 65 or
older
        if (getCustomerAge() >= 65) {
            discount = getMoviePrice() * 0.10;
        }

        double total = getMoviePrice() - discount;


System.out.println("\n---------------------------------"
);
        System.out.println("CUSTOMER: " +
getCustomerName());
        System.out.println("MOVIE: " + getMovieTitle());
        System.out.println("COST: R" + getMoviePrice());
        System.out.println("DISCOUNT: R " + discount);
        System.out.println("TOTAL: R " + total);

System.out.println("---------------------------------");
    }
}
```

4.Movie_Tickets.java

Java

```
import java.util.Scanner;

public class Movie_Tickets {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the customer name: ");
        String name = scanner.nextLine();

        System.out.print("Enter the movie: ");
        String movie = scanner.nextLine();

        System.out.print("Enter the customer age: ");
```

```
        int age = scanner.nextInt();

        System.out.print("Enter the movie cost: ");
        double cost = scanner.nextDouble();

        // Instantiate Subclass
        TicketSales sales = new TicketSales(name, movie,
age, cost);

        // Execute Report
        sales.print_tickets();

        scanner.close();
    }
}
```

SECTION 3: UNIT TESTING TEMPLATE (JUnit 5)

Standard unit test structure for testing array calculations and OOP outputs:

Java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class ProductSalesTest {

    int[][] testData = {
        {300, 700, 150},
        {600, 250, 200}
    };

    @Test
    public void CalculateTotalSales_ReturnsTotalSales() {
        ProductSales sales = new ProductSales();
        int expected = 2200;
        int actual = sales.TotalSales(testData);
        assertEquals(expected, actual);
    }

    @Test
    public void AverageSales_ReturnsAverageProductSales()
{
        ProductSales sales = new ProductSales();
        double expected = 366.6666666666667;
```

```
        double actual = sales.AverageSales(testData);
        assertEquals(expected, actual, 0.001); // 0.001
delta for floating point precision
    }
```

SECTION 1: ARRAY BASICS & 2D ARRAYS

1. 1D Array Declaration & Initialization

Java

```
// Option A: Specific size (default values are 0 for
numbers, null for objects)
int[] numbers = new int[5];

// Option B: Direct initialization
int[] scores = {85, 92, 78, 90, 88};
```

2. 2D Array Traversal (Standard Exam Pattern)

In exam questions (like Question 1), rows usually represent categories/years and columns represent periods/quarters:

Java

```
int[][] quarterlySales = {
    {300, 700, 150}, // Row 0: Year 1
    {600, 250, 200}  // Row 1: Year 2
};

// Nested loop to traverse all elements:
for (int row = 0; row < quarterlySales.length; row++) {
    for (int col = 0; col < quarterlySales[row].length;
col++) {
        System.out.println("Value at [" + row + "][" +
col + "]: " + quarterlySales[row][col]);
    }
}
```

SECTION 2: COMMON ARRAY ALGORITHMS

1. Calculating Total, Average, Max, and Min

Java

```
int total = 0;
```

```
int count = 0;
int max = numbers[0]; // Always initialize max/min with
the FIRST element
int min = numbers[0];

for (int num : numbers) {
    total += num;
    count++;

    if (num > max) {
        max = num;
    }
    if (num < min) {
        min = num;
    }
}

// Cast to double to avoid integer division truncation!
double average = (double) total / count;
```

2. Linear Search

Locating an element in an array and returning its index:

Java

```
public int searchElement(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Found target, return index
        }
    }
    return -1; // Target not found
}
```

SECTION 3: SORTING ALGORITHMS

1. Bubble Sort (Ascending Order)

- - **How it works:** Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Larger elements "bubble up" to the end of the array.
- - **Time Complexity:** O(n2)

Java

```
public class BubbleSortExample {
```

```
    public static void bubbleSort(int[] arr) {
        int n = arr.length;

        // Outer loop for total passes
        for (int i = 0; i < n - 1; i++) {
            // Inner loop for adjacent comparisons
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) { // Change > to
< for descending order
                    // Swap elements using a temporary
variable
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int[] data = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(data);

        System.out.print("Sorted Array (Bubble Sort): ");
        for (int num : data) {
            System.out.print(num + " ");
        }
    }
}
```

2. Insertion Sort (Ascending Order)

- **How it works:** Builds the final sorted array one item at a time. It takes an element from the unsorted portion and shifts elements in the sorted portion to insert it into its correct position.
- **Time Complexity:** O(n2) (Faster than Bubble Sort on small or partially sorted datasets).

Java

```
public class InsertionSortExample {

    public static void insertionSort(int[] arr) {
        int n = arr.length;
```

```
        // Start from index 1 (assume index 0 is already
sorted)
        for (int i = 1; i < n; i++) {
            int key = arr[i]; // Element to insert into
sorted portion
            int j = i - 1;

            // Shift elements of arr[0..i-1] that are
greater than key
            // to one position ahead of their current
position
            while (j >= 0 && arr[j] > key) { // Change >
to < for descending order
                arr[j + 1] = arr[j];
                j = j - 1;
            }
            arr[j + 1] = key; // Place key in its correct
location
        }
    }

    public static void main(String[] args) {
        int[] data = {12, 11, 13, 5, 6};
        insertionSort(data);

        System.out.print("Sorted Array (Insertion Sort):
");
        for (int num : data) {
            System.out.print(num + " ");
        }
    }
}
```

SECTION 4: EXAM QUICK-REF SUMMARY

- **Index Out Of Bounds Exception:** Occurs if you try to access `arr[arr.length]`. Always loop up to `i < arr.length` (strictly less than).
- **Array Swapping Logic:**

Java

```
int temp = arr[a];
```

- `arr[a] = arr[b];`

- - `arr[b] = temp;`